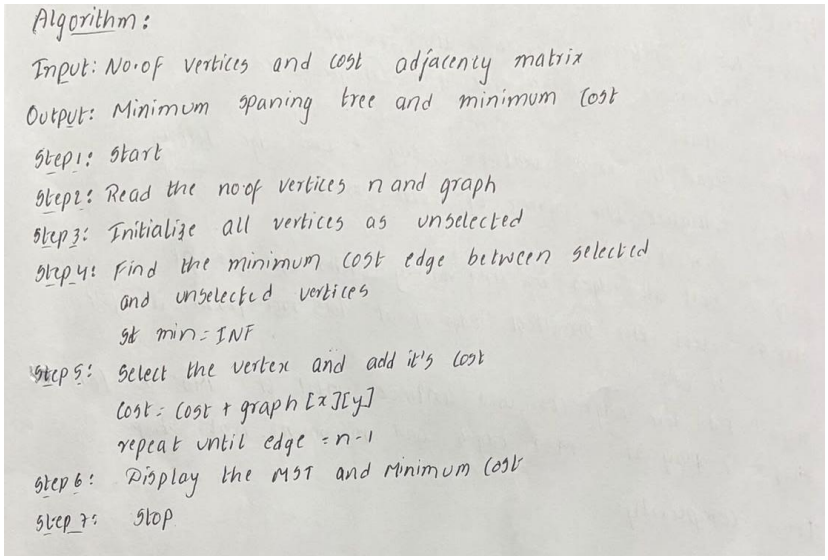
PRACTICAL – 9

AIM: To Implement prim's algorithm

Prim's Algorithm

ALGORITHM:


Algorithm:
Input: No.of vertices and cost adjacency matrix
Output: Minimum spaning tree and minimum cost
Step1: Start
Step2: Read the no.of vertices n and graph
Step3: Initialize all vertices as unselected
Step4: Find the minimum cost edge between selected and unselected vertices
& min = INF
Step5: Select the vertex and add it's cost
cost = cost + graph[x][y]
repeat until edge = n-1
Step6: Display the MST and minimum cost
Step7: Stop

PROGRAM:

```
#include <iostream> using namespace std; int main() {    int graph[MAX][MAX];    int selected[MAX];
  int n, i, j;    int edge = 0;    int x, y;    int min, cost = 0;

  cout << "Enter number of vertices: ";    cin >> n;

  cout << "Enter Cost Adjacency Matrix:\n";

  for(i = 0; i < n; i++)
  {       for(j = 0; j < n; j++)
     {          cin >> graph[i][j];

        if(graph[i][j] == 0)              graph[i][j] = INF;
     }
  }

  // Initially select first vertex    for(i = 0; i < n; i++)       selected[i] = 0;

  selected[0] = 1;
```

```
    cout << "\nEdges in Minimum Spanning Tree:\n";

    while(edge < n - 1)
    {       min = INF;       for(i = 0; i < n; i++)
       {          if(selected[i])
          {             for(j = 0; j < n; j++)
             {
                if(!selected[j] && graph[i][j] < min)
                {                    min = graph[i][j];                    x = i;                    y = j;
                }
             }
          }
       }

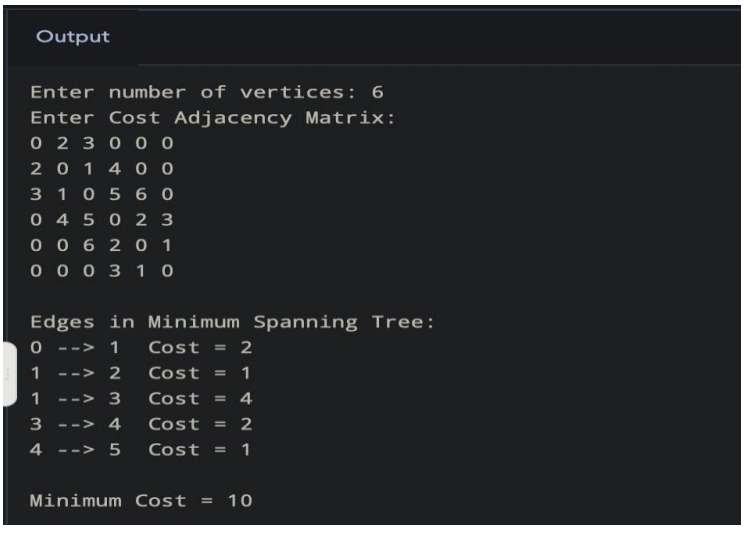
       cout << x << " --> " << y << "  Cost = " << graph[x][y] << endl;

       cost += graph[x][y];       selected[y] = 1;       edge++;
    }

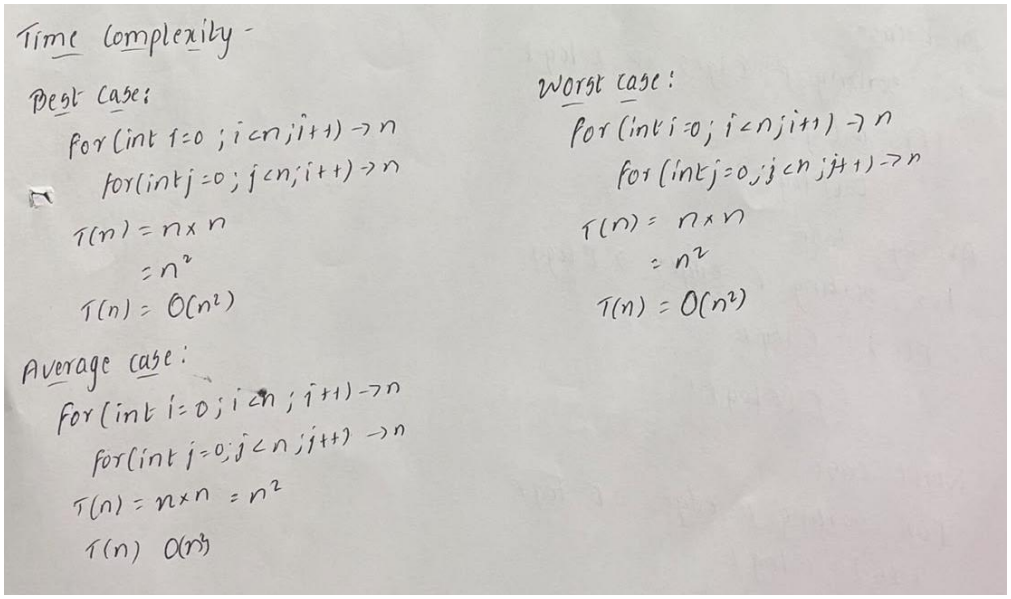
    cout << "\nMinimum Cost = " << cost << endl;

    return 0;
}
```

OUTPUT:



TIME COMPLEXITY:



CONCLUSION:

The implementation of **Prim's Algorithm** helps us understand how a **Minimum Spanning Tree (MST)** is constructed for a weighted graph. It repeatedly selects the minimum-cost edge connecting a selected vertex to an unselected vertex. For the given adjacency matrix implementation, Prim's Algorithm has a time complexity of **$O(n^2)$** in the best, average, and worst cases. Therefore, Prim's Algorithm is a useful technique for finding the minimum-cost spanning tree of a connected graph.